ПРАКТИЧЕСКАЯ РАБОТА № 5
СТРОКИ, МАССИВЫ, ЦИКЛЫ И МЕТОДЫ ОБРАБОТКИ ДАННЫХ В JAVA

ЦЕЛЬ РАБОТЫ

Изучить класс String, метод split с аргументом -1, массивы примитивов int[], циклы for и while, условные операторы и методы с параметрами и возвращаемым значением на прикладной задаче анализа табеля учёта рабочего времени. Освоить строгую валидацию текстового ввода по 9 обязательным правилам R1–R9, стратегию fail-fast и формирование сообщений об ошибках с указанием позиции некорректного значения. Получить работающую программу, которая разбирает строку табеля, вычисляет показатели и обрабатывает ошибки ввода.

Применить полученные знания в индивидуальном сквозном проекте: завершить реализацию starter_project с пятью методами TimesheetAnalyzer (parseHours, total, average, overtimeDays, weeklyOvertime), расширить SelfCheck до 19 unit-проверок и data-driven режима чтения sample.csv, реализовать индивидуальный вариант (часть А – корректировка существующего метода, часть Б – добавление нового метода с предметной логикой). Связать программный код с алгоритмической моделью (Activity-диаграмма) и объектной моделью (диаграмма классов).

СОДЕРЖАНИЕ

1. ПРЕДМЕТНАЯ ПОСТАНОВКА ЗАДАЧИ 2
  1.1. Постановка задачи 2
  1.2. Точный формат входной строки 3
  1.3. Примеры входных строк 3
  1.4. Семантика показателей и бизнес-контекст 4
2. ЗАДАЧИ РАБОТЫ 4
3. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 5
  3.1. Выполнение работы 5
  3.2. Требования к выполнению 6
4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 7
  4.1. Класс String и неизменяемость строк 7
  4.2. Разбор строки через split 7
  4.3. Массивы примитивов в Java 7
  4.4. Циклы for и while 7
  4.5. Условные операторы 8
  4.6. Методы с параметрами и возвращаемым значением 8
  4.7. Валидация текстового ввода и стратегия fail-fast 8
  4.8. Обязательные правила R1–R9 9
  4.9. Data-driven тестирование через sample.csv 9
5. ПРАКТИЧЕСКАЯ ЧАСТЬ. РЕАЛИЗАЦИЯ И ДИАГРАММЫ 9
  5.1. Структура учебного проекта 10
  5.2. Каркас TimesheetAnalyzer с TODO-заглушками 10
  5.3. Правила R1–R9 и связь с кодом 11
  5.4. Заготовка Activity-диаграммы 11
    5.4.1. Назначение 11

5.4.2. Принципы построения Activity-диаграммы 12
Принцип 1. Уровень абстракции – выше кода, ниже «обработать данные» 12
Принцип 2. Не по блоку на каждый оператор 13
Принцип 3. Не один общий блок 13
Принцип 4. Чем отличается от текста в TODO? 13
Принцип 5. Что добавлять для варианта 13
5.4.3. Текстовое описание узлов исходной диаграммы 13
5.4.4. Логика потока 14
5.4.5. Сдача диаграмм 14
5.5. Заготовка диаграммы классов 14
5.6. SelfCheck: 19 обязательных unit-проверок 15
6. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ 16
7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 19
8. ТАБЛИЦА ОЦЕНИВАНИЯ РЕЗУЛЬТАТОВ РАБОТЫ 21
9. ФРАГМЕНТ КОДА ДЛЯ АНАЛИЗА 25
10. КОНТРОЛЬНЫЕ ВОПРОСЫ 26
11. СОДЕРЖАНИЕ ОТЧЁТА 26
11.1. Титульный лист 26
11.2. Разделы отчёта 27
11.3. Требования к оформлению 27
12. ИТОГ 27
ПРИЛОЖЕНИЕ А. РАСШИРЕННЫЙ SAMPLE.CSV 28
ПРИЛОЖЕНИЕ Б. ПРИМЕР ЗАПИСИ ВАРИАНТА В DECISIONS.MD 28
ПРИЛОЖЕНИЕ В. ШАБЛОН ОТЧЁТА 29
ПРИЛОЖЕНИЕ Г. ТРЕБОВАНИЯ К ОТВЕТАМ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 30

1. ПРЕДМЕТНАЯ ПОСТАНОВКА ЗАДАЧИ

Разрабатываемое в практической работе приложение автоматизирует обработку табеля учёта рабочего времени одного сотрудника за рабочую неделю. Предметная областью выступает фрагмент корпоративной системы учёта рабочего времени, в которой данные о часах, отработанных сотрудником за каждый рабочий день (с понедельника по пятницу), поступают в виде текстовой строки и должны быть преобразованы в структурированные показатели, используемые далее для расчёта оплаты труда, контроля нагрузки и формирования управленческой отчётности.

1.1. Постановка задачи

Необходимо обработать строку табеля одного сотрудника за рабочую неделю. В строке через точку с запятой («;») передаются часы за пять рабочих дней – с понедельника по пятницу. Программа должна:

- проверить структуру входной строки (ровно пять значений, разделённых «;»);
- преобразовать значения в целые числа и проверить их соответствие допустимому диапазону и формату;
- вычислить суммарное количество отработанных за неделю часов;
- вычислить среднее количество часов в день с округлением до двух знаков после запятой;

- определить количество дней с переработкой (превышающих дневную норму 8 часов);
- вычислить недельную переработку – превышение общей суммы часов над недельной нормой 40 часов;
- выявить недопустимые значения во входной строке и сформировать понятное сообщение об ошибке с указанием позиции некорректного значения.

Результатом работы программы являются четыре показателя: сумма часов, среднее значение, количество дней с переработкой и недельная переработка. Эти показатели в реальной системе используются отделом кадров для расчёта оплаты труда, руководителем сотрудника для контроля нагрузки и системой аналитики для агрегированной отчётности по подразделению.

1.2. Точный формат входной строки

Входная строка line должна удовлетворять следующим требованиям:

- Разделитель – символ «;» (точка с запятой, код U+003B). Запятая, пробел, табуляция и другие символы не являются разделителями.
- Количество значений – ровно 5. После split(";", -1) массив должен содержать ровно 5 элементов. Меньше или больше – ошибка структуры.
- Формат каждого значения – строковое представление целого числа без ведущих нулей и знака: только строки вида «0», «1», ..., «16». Значения «08», «+8», «8.0», « 8 » (с пробелом) считаются ошибкой формата.
- Surrounding-пробелы не допускаются. Строка «8; 9; 10; 8; 8» (с пробелами после «;») считается ошибочной на позиции 2. Перед разбором не выполнять trim() отдельных значений.
- Trailing/leading разделители не допускаются. Строка «8;9;10;8;8;» (завершающая «;») после split(";", -1) даст 6 элементов, последний – пустой; это нарушает правило «ровно 5 значений».
- Пустые поля не допускаются. Строка «8;;9;10;8» (двойная «;») содержит пустое поле на позиции 2; это трактуется как ошибка формата с указанием позиции.

1.3. Примеры входных строк

Строка	Статус	Пояснение
8;8;8;8;8	Корректна	5 значений, все в диапазоне [0, 16]
9;8;7;10;8	Корректна	5 значений, 2 дня с переработкой (>8)
8;8;17;8;8	Ошибка	Значение 17 на позиции 3 нарушает верхнюю границу
8;abc;8;8;8	Ошибка	Значение abc на позиции 2 не является целым числом
8;8;8;8	Ошибка	4 значения вместо 5 – нарушение структуры
8; 9; 10; 8; 8	Ошибка	Surrounding-пробелы не допускаются (позиция 2)
8;8;8;8;8;	Ошибка	Trailing-разделитель – 6 значений вместо 5
8;;9;10;8	Ошибка	Пустое поле на позиции 2
08;8;8;8;8	Ошибка	Ведущий ноль не допускается (позиция 1)
(пустая строка)	Ошибка	Пустая строка

1.4. Семантика показателей и бизнес-контекст

Показатель	Тип	Бизнес-смысл	Применение
Сумма часов (total)	int	Общее количество отработанных за неделю часов.	Основа для расчёта базовой оплаты; контроль выполнения недельной нормы.
Среднее (average)	double (2 знака)	Среднее количество часов в рабочий день.	Оценка равномерности нагрузки; выявление дней с аномально высокой или низкой нагрузкой.
Дни с переработкой (overtimeDays)	int	Количество дней, в которые сотрудник работал больше дневной нормы (8 часов).	Контроль соблюдения режима труда; основание для дополнительной оплаты за переработку.
Недельная переработка (weeklyOvertime)	int	Превышение общей суммы часов над недельной нормой (40 часов); не может быть отрицательным.	Основа для расчёта оплаты сверхурочной работы; индикатор риска выгорания.

Дневная и недельная переработки – две независимые метрики. Дневная показывает, в скольких днях сотрудник превысил норму (R5); недельная показывает, на сколько часов суммарно сотрудник превысил недельную норму (R6). Они не суммируются и не агрегируются в единый показатель: для строки «9;9;9;9;9» получаем overtimeDays = 5 (все дни) и weeklyOvertime = 5 (45 − 40). Эти два числа не связаны прямой формулой и анализируются независимо.

Именно для решения этой прикладной задачи в работе реализуются класс TimesheetAnalyzer с пятью методами, расширение SelfCheck с unit- и data-driven проверками, индивидуальные варианты (часть А изменяет бизнес-правило, часть Б добавляет новый показатель), Activity-диаграмма алгоритма обработки и диаграмма классов объектной модели.

2. ЗАДАЧИ РАБОТЫ

В ходе выполнения практической работы необходимо:

- актуализировать знания о классе String, методе split, массивах примитивов и циклах Java;
- освоить строгий разбор строки табеля одного сотрудника за рабочую неделю: строка вида «8;9;10;8;8» преобразуется в массив из 5 целых чисел в диапазоне [0, 16];
- реализовать класс TimesheetAnalyzer с пятью public static методами: parseHours, total, average, overtimeDays, weeklyOvertime – без изменения сигнатур;
- применить 9 обязательных правил обработки (R1–R9): проверка количества значений, regex формата, диапазона, семантика значения 0, дневная и недельная пе-

реработка, fail-fast стратегия, нумерация позиций с 1, классификация ошибок формата и диапазона;

- реализовать стратегию fail-fast: при обнаружении первой ошибки валидация прекращается и выбрасывается IllegalArgumentException с сообщением, содержащим позицию и описание;
- расширить класс SelfCheck до минимума 19 unit-проверок с распределением по методам (parseHours – 7, total – 3, average – 3, overtimeDays – 3, weeklyOvertime – 3);
- реализовать data-driven режим SelfCheck: чтение файла data/sample.csv, разбор строк формата <вход>|<ожидание>|<комментарий>, прогон всех тест-кейсов без перекомпиляции;
- реализовать интерактивный режим для защиты: чтение строк из System.in, обработка до ввода exit или пустой строки;
- завершить Activity-диаграмму (блоки TODO A1 и A2) с конкретными формулировками действий, соблюдая пять принципов уровня детализации;
- завершить диаграмму классов: отразить все public-методы TimesheetAnalyzer;
- выбрать индивидуальный вариант и реализовать обе части: А – корректировка существующего метода, Б – добавление нового метода с предметной логикой;
- адаптировать SelfCheck и sample.csv под выбранный вариант: добавить минимум 2 проверки для части А и 2 для части Б;
- подготовить файл DECISIONS.md (80–150 строк) с описанием варианта, двух принципиальных решений, ответов на 4 контрольных вопроса и анализа фрагмента кода с ошибкой off-by-one;
- зафиксировать изменения в системе контроля версий Git с меткой PR5: и разместить публичный репозиторий на GitHub;
- подготовить отчёт в формате .docx или .pdf (5–10 страниц) и защитить работу длительностью 10–15 минут.

3. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

Работа выполняется в существующем проекте starter_project, поставляемом преподавателем. Проект компилируется, но намеренно содержит незавершённые места, помеченные TODO. Создание нового проекта не допускается. Изменения выполняются в существующей пакетной структуре src/ru/edu/pr02/.

3.1. Выполнение работы

Практическая часть выполняется последовательно: изучение теории (раздел 4), реализация 5 методов, расширение SelfCheck, завершение диаграмм. Последовательность доработки проекта:

- Изучить структуру `starter_project` и определить место каждого TODO в общей архитектуре.
- Реализовать `parseHours(String line)` – единственную точку валидации пользовательского ввода по правилам R1–R9.
- Реализовать `total(int[] hours)` – сумму элементов с защитой от null и неверной длины.
- Реализовать `average(int[] hours)` – среднее с двумя знаками после запятой, использующее `total(hours)`.
- Реализовать `overtimeDays(int[] hours)` – количество дней со значением v > 8 (строгое неравенство).
- Реализовать `weeklyOvertime(int[] hours) - max(0, total(hours) - 40)`, использующее `total(hours)`.
- Расширить `SelfCheck` до 19 unit-проверок с распределением по методам (подраздел 5.6).
- Реализовать data-driven режим: чтение data/sample.csv, разбор строк, прогон тест-кейсов.
- Реализовать интерактивный режим для защиты: чтение из System.in до exit или пустой строки.
- Завершить Activity-диаграмму: блоки TODO A1 и A2 с конкретными формулировками (подраздел 5.4).
- Завершить диаграмму классов: отразить все public-методы TimesheetAnalyzer (подраздел 5.5).
- Выбрать индивидуальный вариант и реализовать обе части: А (корректировка) и Б (новый метод).
- Адаптировать SelfCheck и sample.csv под вариант: +4 проверки и 3–5 строк данных.
- Подготовить DECISIONS.md, отчёт и защиту (разделы 7, 8, 9).

3.2. Требования к выполнению

- сохранить пакетную структуру src/ru/edu/pr02/ и не изменять сигнатуры 5 методов TimesheetAnalyzer (кроме случаев, когда выбранный вариант явно требует);
- не использовать Stream API (java.util.stream), включая Arrays.stream, IntStream, Stream.of;
- не использовать лямбда-выражения и method references (::) в любом контексте;
- не использовать Iterable.forEach, List.forEach для итерации и внешние библиотеки (Apache Commons, Guava);
- разрешены: классические циклы for/while/do-while, enhanced for-each, var, switch expressions, text blocks, утилитные методы Math, Integer, String;

- не создавать отдельный класс Main/App – main уже есть в SelfCheck;
- обязательно использовать split(";", -1) (а не split(";")) для сохранения trailing-пустых строк;
- фиксировать изменения в Git с меткой PR5: и поддерживать актуальность README.md.

4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

4.1. Класс String и неизменяемость строк

В языке Java строки представлены классом java.lang.String. Объекты String являются неизменяемыми (immutable): любая операция, выглядящая как изменение строки, создаёт новый объект. Метод split возвращает новый массив строк, не изменяя исходный объект. Сравнение строк на равенство содержимого выполняется методом equals, а не оператором ==, который сравнивает ссылки.

4.2. Разбор строки через split

Метод `String.split(String regex, int limit)` разделяет строку на массив подстрок. Для табеля используется разделитель «;». Параметр limit = -1 сохраняет trailing-пустые строки; limit = 0 (по умолчанию) их удаляет. Для проверки правила R1 «ровно 5 значений» обязательно использовать `split(";", -1)`. Если бы разделителем выступал символ «.» или «|», его потребовалось бы экранировать: `split("\\.")` или `split("\\|")`.

```
String input = "8;9;10;8;8";
String[] parts = input.split(";", -1); // аргумент -1 обязателен
if (parts.length != 5) {
    throw new IllegalArgumentException(
        "Ошибка: структура -- ожидалось 5 значений, получено " + parts.length);
}
```

4.3. Массивы примитивов в Java

Массив в Java – это объект, хранящий фиксированное количество значений одного типа. Длина массива доступна через поле length (без скобок). Индексация начинается с нуля, индекс последнего элемента равен length − 1. При обращении к элементу по индексу за пределами массива выбрасывается `ArrayIndexOutOfBoundsException`. Преобразование строки в число выполняется методом `Integer.parseInt(String);` при неверном формате выбрасывается `NumberFormatException`.

4.4. Циклы for и while

В Java применяются четыре формы циклов: классический `for` с индексом, расширенный `for-each`, цикл `while` с предусловием и цикл `do-while` с постусловием. В данной работе для обхода массива часов используется классический `for`, поскольк-

ку индекс требуется для формирования сообщений об ошибках и для обращения к конкретному дню недели. Цикл `for-each` в этом случае неприменим: он не предоставляет индекса элемента. Цикл `while` уместен при чтении строк из файла sample.csv в data-driven режиме SelfCheck. Stream API (Arrays.stream, IntStream) в данной работе запрещён.

4.5. Условные операторы

Условные операторы `if`, `if-else`, `if-else if-else` и `switch` управляют потоком выполнения. В задаче табеля условные операторы применяются для проверки значения каждого дня: если v > 8, день считается переработкой; если v < 0 или v > 16 – значение некорректно. Тернарный оператор ? : используется для краткой записи простых альтернатив, например для вычисления недельной переработки: max(0, total − 40) реализуется как `Math.max(0, total - 40)`.

4.6. Методы с параметрами и возвращаемым значением

Все 5 методов TimesheetAnalyzer – public static, без полей класса (stateless). Метод average(int[] hours) должен вызывать total(hours), а weeklyOvertime(int[] hours) должен вызывать total(hours) – это исключает дублирование логики суммирования (принцип DRY). Защитные условия в методах вычислений – это программные контракты: проверка hours == null и hours.length != 5 с выбросом IllegalArgumentException. Проверка диапазона значений (0 ≤ v ≤ 16) в методах вычислений не требуется: если массив получен из parseHours, диапазон уже гарантирован.

```
public static int total(int[] hours) {
    int total = 0;
    for (int h : hours) total += h;
    return total;
}

public static double average(int[] hours) {
    return (double) total(hours) / hours.length;
}

public static int overtimeDays(int[] hours) {
    int count = 0;
    for (int h : hours) if (h > 8) count++;
    return count;
}
```

4.7. Валидация текстового ввода и стратегия fail-fast

Валидация входных данных в данной работе выполняется по 9 обязательным правилам R1–R9. Стратегия обработки ошибок – fail-fast: при обнаружении первой ошибки валидация прекращается и выбрасывается `IllegalArgumentException` с сообщением, содержащим позицию (с 1) и описание ошибки. Сообщение должно соответствовать шаблону «Ошибка: позиция <N> -- <краткое описание>».

Классификация ошибок (правило R9) критически важна: дробные значения (8.5) классифицируются как ошибка формата «не целое число», а не как нарушение диапазона; отрицательные значения (-3) – ошибка формата (нарушение regex), а не нарушения диапазона. Это различие обязательно проверяется в SelfCheck. Окружающие пробелы (8; 9; 10; 8; 8) считаются ошибкой формата на позиции 2; trim() отдельных значений не выполняется – это намеренное решение для строгой валидации.

4.8. Обязательные правила R1–R9

№	Правило	Точная формулировка
R1	Количество значений	После split(";", -1) ровно 5 элементов. Иначе – ошибка структуры.
R2	Тип значения	Каждый элемент соответствует regex ^(0|[1-9][0-9]*)$ – целое без ведущих нулей и знака.
R3	Диапазон	$0 \le v \le 16$ включительно. Границы 0 и 16 допустимы.
R4	Семантика 0	0 трактуется как выходной; корректен, участвует в расчётах.
R5	Дневная переработка	День считается переработкой при $v > 8$ (строго). Значение 8 не является.
R6	Недельная переработка	max(0, sum − 40), где sum – сумма всех 5 значений. Независима от R5
R7	Стратегия ошибок	Fail-fast: первая ошибка прекращает валидацию, выбрасывается IllegalArgumentException.
R8	Нумерация позиций	С 1 (первый день = позиция 1).
R9	Классификация	Дробные (8.5) и отрицательные (-3) – ошибка формата (нарушение regex), не диапазона.

4.9. Data-driven тестирование через sample.csv

Файл data/sample.csv содержит контрольные значения для тестирования. Формат каждой строки: `<вход>|<ожидание>|<комментарий>`. Поле `<вход>` – строка табеля или EMPTY для пустой строки. Поле `<ожидание>` – либо `OK:sum=S,avg=A.AA,od=O,wo=W` для корректной строки с ожидаемыми метриками, либо `ERROR:позиция N / ERROR:структура` для ожидаемой ошибки. Data-driven режим SelfCheck читает файл, для каждой строки вызывает parseHours и остальных методы, сравнивает с ожиданием, выводит `PASS data/N` или `FAIL data/N: <причина>`.

5. ПРАКТИЧЕСКАЯ ЧАСТЬ. РЕАЛИЗАЦИЯ И ДИАГРАММЫ

Практическая часть связывает теоретические сведения раздела 4 с конкретной реализацией. Требуется последовательно выполнить следующие действия:

- изучает структуру starter_project (5.1);
- читает каркас TimesheetAnalyzer с TODO-заглушками и подсказками (5.2);
- реализует 5 TODO по правилам R1–R9 (5.3);
- сверяет реализацию с заготовкой Activity-диаграммы (5.4) и диаграммы классов (5.5);
- расширяет SelfCheck до 19 unit-проверок (5.6).

Каждой задаче кода соответствует теоретический раздел: String и split – 4.1, 4.2; массивы – 4.3; циклы – 4.4; методы и DRY – 4.6; валидация и fail-fast – 4.7.

5.1. Структура учебного проекта

```
starter_project/
├── README.md
├── run_selfcheck.sh            # Unix-скрипт запуска
├── run_selfcheck.bat           # Windows-скрипт запуска
├── data/
│   └── sample.csv              # Контрольные значения
├── src/
│   └── ru/edu/pr02/
│       ├── TimesheetAnalyzer.java   # 5 методов с TODO
│       └── SelfCheck.java          # main и тесты
├── activity_partial.dot        # Заготовка Activity-диаграммы
└── class_partial.dot           # Заготовка диаграммы классов
```

5.2. Каркас TimesheetAnalyzer с TODO-заглушками

Класс TimesheetAnalyzer содержит 5 public static методов. Ниже приведён каркас класса с TODO-заглушками. Каждая заглушка содержит: сигнатуру метода; правила R1–R9, которые нужно реализовать; подсказки по реализации со ссылками на теоретические разделыНеобходимо заменить каждый TODO на конкретный код в соответствии со спецификацией.

```
package ru.edu.pr02;

public class TimesheetAnalyzer {
    // TODO 1: parseHours(String line) → int[]
    //   R1: split(";", -1), проверить parts.length == 5, иначе IAE("Ошибка: структура -- ...")
    //   R2: regex ^(0|[1-9][0-9]*)$ для каждого элемента, иначе IAE("Ошибка: позиция N -- ...")
    //   R3: 0 <= v <= 16, иначе IAE("Ошибка: позиция N -- вне диапазона [0, 16]")
    //   R7: fail-fast – при первой ошибке выброс исключения без продолжения
    //   R8: позиции с 1 (i + 1 в сообщении)
    //   R9: дробные и отрицательные – ошибка формата (regex), а не диапазона
    //   Подсказка: используйте split(";", -1), matches(), Integer.parseInt()
    public static int[] parseHours(String line) {
        // TODO: реализуйте проверки R1-R9 и преобразование в int[]
        return new int[0];
    }

    // TODO 2: total(int[] hours) → int
    //   Защитные условия: hours == null → IAE, hours.length != 5 → IAE
    //   Сумма элементов через цикл for (без Stream API)
    public static int total(int[] hours) {
        // TODO: реализуйте сумму
        return 0;
    }

    // TODO 3: average(int[] hours) → double
    //   Защитные условия аналогичны total
    //   Округление до 2 знаков: Math.round(value * 100.0) / 100.0
    //   Использует total(hours) – DRY
```

```
    public static double average(int[] hours) {
        // TODO: реализуйте среднее с 2 знаками
        return 0.0;
    }

    // TODO 4: overtimeDays(int[] hours) → int
    //   Защитные условия аналогичны total
    //   Подсчёт дней с v > 8 (строгое неравенство, R5)
    public static int overtimeDays(int[] hours) {
        // TODO: реализуйте подсчёт
        return 0;
    }

    // TODO 5: weeklyOvertime(int[] hours) → int
    //   Защитные условия аналогичны total
    //   Math.max(0, total(hours) - 40) – DRY (использует total)
    public static int weeklyOvertime(int[] hours) {
        // TODO: реализуйте недельную переработку
        return 0;
    }
}
```

5.3. Правила R1–R9 и связь с кодом

Правила R1–R9 (таблица в разделе 4.8) – это контракт класса TimesheetAnalyzer. Каждое правило реализуется в конкретном методе и проверяется unit-тестом. После реализации каждого TODO сверьтесь с таблицей: проверьте, что правило реализовано в нужном методе, выбрасывается правильное исключение (IAE для данных, ISE для переходов), сообщение содержит контекст.

5.4. Заготовка Activity-диаграммы

Activity-диаграмма визуализирует алгоритм обработки строки табеля: последовательность проверок и ветвлений. Узлы – этапы обработки (получение строки, split, проверка длины, цикл проверок значений, вычисления, формирование отчёта). Рёбра – переходы между этапами. Студент должен завершить диаграмму, добавив конкретные формулировки в блоки TODO A1 и A2.

5.4.1. Назначение

На рисунке 1 дан каркас алгоритма. Красные блоки TODO A1 и A2 намеренно оставлены неполными. Студент должен сформулировать действие точнее так, чтобы последовательность ветвлений совпадала с реализацией. Формулировка – конкретная проверка или преобразование, не общая фраза «обработать данные».

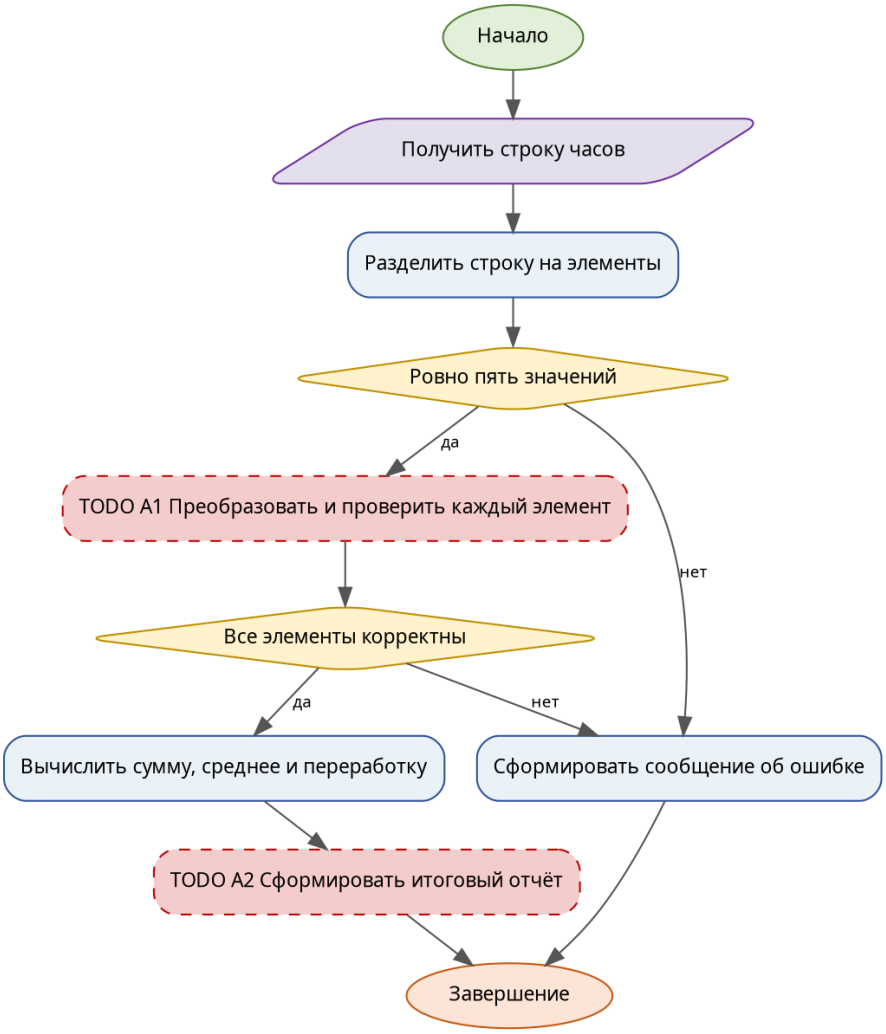


Рисунок 1 – Заготовка Activity-диаграммы с блоками TODO A1 и A2

5.4.2. Принципы построения Activity-диаграммы

Activity-диаграмма показывает алгоритм обработки строки табеля – последовательность действий и ветвлений. Правильный уровень детализации – ключевой методический вопрос. Сформулируем принципы:

Принцип 1. Уровень абстракции – выше кода, ниже «обработать данные»

Каждый блок диаграммы должен описывать одно логически завершённое действие, имеющее смысл как самостоятельный шаг алгоритма. Это не копия кода (где был бы отдельный блок на каждый оператор) и не общая фраза (где был бы один блок «обработать данные»).

Уровень	Пример блока	Допустимость
Слишком низко (копия кода)	i++; if (i < 5) continue;	Недопустимо – это код, а не алгоритм
Правильно	«Проверить каждый элемент: regex и диапазон [0, 16]»	Допустимо – логический шаг
Слишком высоко	«Обработать данные»	Недопустимо – не информирует

Принцип 2. Не по блоку на каждый оператор

Если в коде метод parseHours содержит 15 строк (создание массива, цикл, проверка regex, parseInt, проверка диапазона, присваивание), на диаграмме это не 15 блоков. Цикл и проверки внутри него – это один-два блока: «Проверить каждый элемент» (можно разделить на «Проверить формат» и «Проверить диапазон», если эти шаги концептуально различны для автора).

Принцип 3. Не один общий блок

Свести всю обработку к одному блоку «Обработать строку» – бесполезно. Диаграмма должна показывать ветвления (где принимаются решения) и последовательность существенных шагов. Если в коде есть if/else, на диаграмме должно быть ветвление (ромб). Если в коде есть цикл, на диаграмме должен быть виден цикл (через back-edge или пометку «для каждого элемента»).

Принцип 4. Чем отличается от текста в TODO?

TODO в исходной диаграмме – это метка незавершённости, указывающая, что блок нужно заполнить. Когда студент завершает блок, он заменяет общую формулировку TODO на конкретное действие, отражающее реализацию. Уровень детализации заполненного блока – тот же, что и у остальных блоков диаграммы (не мельче, не крупнее).

Блок TODO	Недопустимое заполнение	Допустимое заполнение
TODO A1: Преобразовать и проверить каждый элемент	«Преобразовать i-й элемент в int через parseInt; проверить regex ^(0|[1-9][0-9]*)$; проверить 0 ≤ v ≤ 16; присвоить result[i] = v; инкрементировать i»	«Для каждого элемента: проверить regex и диапазон [0, 16], преобразовать в int»
TODO A2: Сформировать итоговый отчёт	«System.out.println(\"Сумма: \" + sum); System.out.println(\"Среднее: \" + avg); ...»	«Сформировать строку: сумма, среднее (2 знака), overtimeDays, weeklyOvertime»

Принцип 5. Что добавлять для варианта

Если выбранный вариант (раздел 6) изменяет логику существующего метода, на диаграмме не нужно добавлять новые блоки – достаточно обновить формулировку существующего блока. Если вариант добавляет новый метод (часть Б варианта), добавляется один новый блок. Не нужно добавлять блоки на каждый оператор внутри нового метода.

5.4.3. Текстовое описание узлов исходной диаграммы

Activity-диаграмма (activity_partial.dot) содержит следующие узлы (в порядке потока управления):

Узел	Тип	Надпись	Действие студента
start	Начало	«Начало»	Без изменений
n1	Ввод	«Получить строку часов»	Без изменений
n2	Действие	«Разделить строку на элементы»	Уточнить: split(";", -1)
n3	Решение	«Ровно пять значений?»	Без изменений

Узел	Тип	Надпись	Действие студента
n4	TODO A1	«Преобразовать и проверить каждый элемент»	Уточнить: для каждого элемента – regex + диапазон [0, 16], parseInt
n5	Решение	«Все элементы корректны?»	Без изменений
n6	Действие	«Вычислить сумму, среднее и переработку»	Уточнить: total, average (2 знака), overtimeDays, weeklyOvertime
n7	TODO A2	«Сформировать итоговый отчёт»	Уточнить: собрать строку с 4 показателями
n8	Действие	«Сформировать сообщение об ошибке»	Без изменений
end	Завершение	«Завершение»	Без изменений

5.4.4. Логика потока

Поток управления: start → n1 → n2 → n3. Если «нет» (не 5 значений) – переход к n8 (сообщение об ошибке). Если «да» – переход к n4 (TODO A1: валидация и преобразование). После n4 → n5: если «нет» (не все корректны) – к n8; если «да» – к n6 (вычисления). Затем n6 → n7 (TODO A2: отчёт) → end. Из n8 → end. На защите студент должен показать путь исполнения для конкретного входа: например, для строки «8;8;8;8;8» путь start → n1 → n2 → n3(да) → n4 → n5(да) → n6 → n7 → end; для строки «8;8;17;8;8» путь start → n1 → n2 → n3(да) → n4 → n5(нет) → n8 → end.

5.4.5. Сдача диаграмм

Студент сдаёт отредактированные activity.dot. Допускается использование Graphviz DOT или любого UML-редактора (draw.io, PlantUML, Visual Paradigm, dreampuf) с экспортом в PNG.

5.5. Заготовка диаграммы классов

На рисунке 2 – ожидаемая структура решения. Класс TimesheetAnalyzer помечен TODO: студент должен завершить методы. Если добавляете вспомогательные классы или новые методы (часть Б варианта) – отразите на диаграмме.

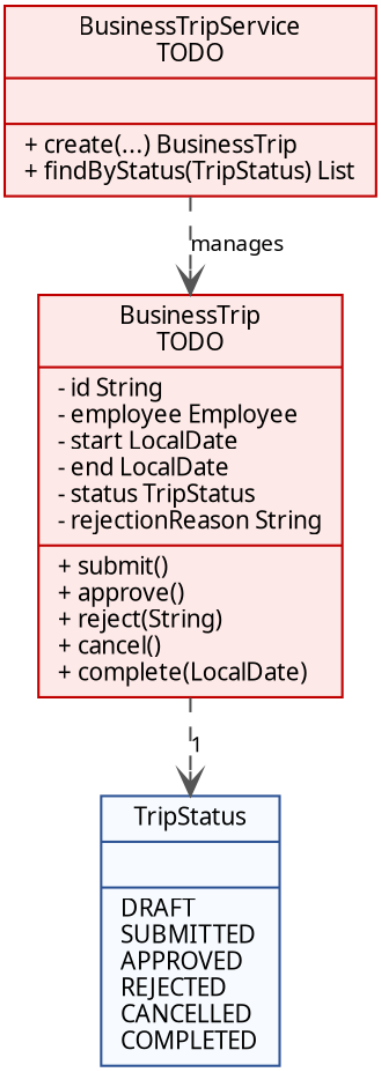


Рисунок 2 – Заготовка диаграммы классов

5.6. SelfCheck: 19 обязательных unit-проверок

SelfCheck – это тестовый раннер: вызывает методы TimesheetAnalyzer на заранее подготовленных тестовых случаях и сравнивает результат с ожиданием. Минимальный набор – 19 unit-проверок, распределённых по правилам R1–R9 и методам. Каждая проверка соответствует одному правилу или сценарию; это обеспечивает покрытие всех TODO и инвариантов.

№	Метод	Сценарий	Вход	Ожидание
1	parseHours	Корректная строка	8;9;10;8;8	{8,9,10,8,8}
2	parseHours	Пустая строка	(пустая)	IAE: структура
3	parseHours	Значение 17	8;8;17;8;8	IAE: позиция 3
4	parseHours	Не-число	8;abc;8;8;8	IAE: позиция 2
5	parseHours	4 значения	8;8;8;8	IAE: структура
6	parseHours	Trailing ;	8;8;8;8;8;	IAE: структура
7	parseHours	Ведущий ноль	08;8;8;8;8	IAE: позиция 1
8	total	Типовое	{8,9,10,8,8}	43
9	total	Нижняя граница	{0,0,0,0,0}	0
10	total	Верхняя граница	{16,16,16,16,16}	80

№	Метод	Сценарий	Вход	Ожидание
11	average	Типовое	{8,9,10,8,8}	8.6
12	average	Целое	{8,8,8,8,8}	8.0
13	average	Дробное	{7,8,9,8,7}	7.8
14	overtimeDays	Нет переработки	{8,8,8,8,8}	0
15	overtimeDays	Два дня	{9,8,7,10,8}	2
16	overtimeDays	Граница 8	{8,9,8,8,8}	1
17	weeklyOvertime	Сумма 40	{8,8,8,8,8}	0
18	weeklyOvertime	Сумма 50	{10,10,10,10,10}	10
19	weeklyOvertime	Сумма < 40	{6,6,6,6,6}	0

Data-driven режим: чтение data/sample.csv формата `<вход>|<ожидание>|<комментарий>`; для каждой строки вызов parseHours и остальных методов; PASS data/N или FAIL data/N: <причина>. Интерактивный режим: чтение строк из System.in до exit или пустой строки; вывод суммы, среднего (2 знака), overtimeDays, weeklyOvertime.

6. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ

Номер варианта – порядковый номер студента в группе. Каждый вариант состоит из двух частей:

- А – корректировка существующего правила или метода);
- Б – добавление нового public static метода с предметной логикой.

По результатам выполнения задания своего варианта необходимо адаптировать SelfCheck: 19 базовых проверок остаются (с обновлёнными ожиданиями), добавить минимум 2 проверки для части А и 2 для части Б.

№	Часть А	Часть Б
1	average возвращает медиану (3-й элемент отсортированного массива из 5 чисел). Использовать Arrays.sort для копии массива.	`addProductivityLevel(int[] hours) → String` – возвращает текстовую оценку продуктивности за неделю: «низкая» если среднее < 6 часов, «средняя» если среднее в диапазоне от 6 до 9 часов, «высокая» если среднее ≥ 9 часов. Использует результат метода `average`.
2	overtimeDays возвращает размах (max − min), а не количество дней с v > 8.	`addWorkloadScore(int[] hours) → int` – вычисляет интегральную оценку загрузки сотрудника за неделю: сумма отработанных часов (`total`) плюс удвоенное количество дней с переработкой (`overtimeDays` × 2). Чем выше оценка, тем больше нагрузка.
3	weeklyOvertime возвращает max(0, 40 − sum) – количество «недоработанных» часов до нормы.	`addRestHours(int[] hours) → int` – возвращает количество часов отдыха за неделю: 5 × 8 − `total`, но не менее 0. Если сотрудник отработал больше нормы, возвращает 0. Показывает, сколько часов не дотянул до стандартной 40-часовой недели.
4	total возвращает сумму только ненулевых значений (дни с 0 = выходные, исключаются).	`addWorkingDaysCount(int[] hours) → int` – возвращает количество фактически отработанных дней: число элементов массива, значение которых больше 0. Дни с 0 (выходные) не учитываются.

№	Часть А	Часть Б
5	average возвращает взвешенное среднее с весами [1.0, 1.0, 1.2, 1.0, 0.8] (среда важнее, пятница легче).	`addWeightedScore(int[] hours) → double` – вычисляет взвешенную сумму часов с весами по дням недели: понедельник 1.0, вторник 1.0, среда 1.2, четверг 1.0, пятница 0.8. Каждый элемент массива умножается на соответствующий вес, результаты суммируются.
6	overtimeDays считает переработкой v ≥ 8 (включительно), а не v > 8.	`addOvertimeHours(int[] hours) → int` – возвращает суммарное количество переработанных часов: для каждого дня, где значение больше 8 (или ≥ 8 согласно части А), прибавляет (v − 8) к результату. Показывает общий объём переработки в часах.
7	weeklyOvertime: прогрессивная шкала – если sum > 48, = (sum − 48) × 2; если 40 < sum ≤ 48, = sum − 40; иначе 0.	`addFatigueIndex(int[] hours) → int` – вычисляет индекс усталости сотрудника: количество дней с переработкой (`overtimeDays`) умножить на 3, и если общая сумма часов больше 40, добавить 2. Чем выше индекс, тем выше риск выгорания.
8	parseHours принимает дробные значения (напр. 8.5), возвращает double[]. Все остальные методы переключить на double[].	`addHalfDaysCount(double[] hours) → int` – возвращает количество полноценных рабочих дней, где отработано 4 и более часов. Значения меньше 4 считаются половиной дня и не учитываются. Параметр `double[]` – результат изменённого `parseHours` из части А.
9	parseHours принимает значения с ведущими нулями (08 = 8). Regex меняется на ^0*[0-9]+$ с проверкой диапазона.	`addFormattedTimesheet(int[] hours) → String` – формирует человекочитаемую строку табеля вида «Пн:8, Вт:9, Ср:10, Чт:8, Пт:8» из массива часов. Удобно для вывода в отчёт или лог.
10	parseHours выполняет trim() каждого значения после split. Surrounding-пробелы допускаются.	`addIsValidStructure(int[] hours) → boolean` – проверяет структуру недели: возвращает `true`, если в массиве нет двух подряд идущих дней со значением 0 (два выходных подряд считаются подозрительными). Иначе `false`.
11	parseHours принимает значения до 24 включительно. Диапазон R3 меняется на 0 ≤ v ≤ 24.	`addNightHours(int[] hours) → int` – возвращает количество ночных смен: число дней, где отработано 20 и более часов. Используется для контроля переработки и охраны труда.
12	parseHours ожидает 7 значений (пн–вс). R1 меняется на «ровно 7». Остальные методы работают с int[7].	`addWeekendHours(int[] hours) → int` – возвращает сумму часов, отработанных в выходные дни (позиции 6 и 7 массива). Позволяет отдельно учесть работу в субботу и воскресенье.
13	parseHours игнорирует пустые значения на концах. Trailing ; допускается.	`addMinWorkday(int[] hours) → int` – возвращает минимальное ненулевое значение в массиве (самый короткий рабочий день). Если все значения 0, возвращает 0. Показывает минимальную нагрузку среди отработанных дней.
14	parseHours отбрасывает часть строки после символа # (комментарий).	`addHasOvertimeOnlyAtEnds(int[] hours) → boolean` – возвращает `true`, если переработки зафиксированы только в первом и последнем дне недели (позиции 1 и 5), а в средних днях (позиции 2, 3, 4) переработок нет. Иначе `false`.
15	weeklyOvertime возвращает max(0, sum − 36) (недельная норма 36 часов, не 40).	`addOvertimePay(int[] hours, double rate) → double` – вычисляет оплату переработки: недельная переработка (`weeklyOvertime`) умножается на почасовую ставку `rate` и на коэффициент 1.5. Возвращает сумму в рублях.

№	Часть А	Часть Б
16	overtimeDays считает переработкой v > 10 (норма 10 часов).	`addHeavyDaysCount(int[] hours) → int` – возвращает количество тяжёлых смен: число дней, где отработано более 12 часов. Используется для выявления чрезмерных нагрузок.
17	weeklyOvertime учитывает не более 3 дней с максимальной переработкой. Если overtimeDays > 3, из суммы вычитаются только переработки 3 самых «переработанных» дней.	`addTop3Overtime(int[] hours) → int` – возвращает сумму превышений над 8 часами для трёх дней с наибольшей переработкой. Если дней с переработкой меньше трёх, учитываются все. Алгоритм: найти превышения, отсортировать по убыванию, взять первые 3, просуммировать.
18	average делит на количество ненулевых дней, а не на 5. Если все 0 – возвращает 0.	`addZeroDaysCount(int[] hours) → int` – возвращает количество дней со значением 0 (выходные, больничные, отгулы). Показывает, сколько дней сотрудник не работал.
19	total возвращает сумму квадратов значений (не сумму).	`addVariance(int[] hours) → double` – вычисляет выборочную дисперсию: для каждого значения `v` вычисляется отклонение от среднего (`v − avg`), возводится в квадрат, результаты суммируются и делятся на 5. Показывает разброс значений вокруг среднего.
20	overtimeDays возвращает сумму превышений над 8 (не количество дней).	`addMaxOvertimeDay(int[] hours) → int` – возвращает максимальное превышение над 8 часами среди всех дней. Если переработок нет (все `v ≤ 8`), возвращает 0. Показывает самый напряжённый день.
21	parseHours принимает 0 как больничный. Остальные методы исключают 0 из расчётов.	`addSickDaysCount(int[] hours) → int` – возвращает количество дней со значением 0, которые трактуются как больничные или выходные. Используется для учёта пропусков.
22	overtimeDays считает переработкой v > 8 только в позициях 1 и 5 (крайние дни). В позициях 2–4 переработка не считается.	`addMiddleDaysSum(int[] hours) → int` – возвращает сумму часов, отработанных в середине недели (вторник, среда, четверг – позиции 2, 3, 4 массива). Позволяет отдельно анализировать середину недели.
23	weeklyOvertime = сумма превышений над 8 для позиций 1 и 5 (крайние дни). Остальные дни не учитываются.	`addEndsOvertime(int[] hours) → int` – возвращает сумму превышений над 8 часами для крайних дней недели (понедельник – позиция 1, пятница – позиция 5). Для каждого дня вычисляется `max(0, v - 8)`, результаты суммируются.
24	average возвращает среднее без учёта минимального и максимального значения (обрезанное среднее по 3 средним элементам).	`addRange(int[] hours) → int` – возвращает размах: разность между максимальным и минимальным значениями в массиве. Показывает вариативность нагрузки в течение недели.
25	weeklyOvertime = сумма (v − 8) × 1.5 для v > 8 (переработка в 1.5 раза). Возвращать int через (int) Math.round(...).	`addOvertimeCost(int[] hours, double hourlyRate) → double` – вычисляет стоимость переработки: для каждого дня с переработкой (`v > 8`) вычисляется `(v − 8) × 1.5 × hourlyRate`, результаты суммируются. Возвращает сумму в рублях.
26	parseHours ожидает 6 значений: 5 часов + ставка (double). Возвращает int[5] (первые 5).	`addPayForWeek(int[] hours, double rate) → double` – вычисляет полную оплату за неделю: обычная оплата (`total × rate`) плюс оплата переработки (`weeklyOvertime × rate × 1.5`). Возвращает сумму в рублях. Параметр `rate` – почасовая ставка.

№	Часть А	Часть Б
27	parseHours принимает −1 как «отгул». overtimeDays не считает отгулы. average делит на количество неотгульных дней.	`addDayOffCount(int[] hours) → int` – возвращает количество отгулов: число дней со значением −1 (согласно части А, `parseHours` принимает −1 как отгул). Показывает, сколько дней сотрудник взял отгулов.
28	weeklyOvertime комбинирует: если overtimeDays ≤ 3, = sum − 40. Если > 3, = (сумма превышений топ-3 дней) × 2.	`addOvertimeCategory(int[] hours) → String` – возвращает текстовую категорию объёма переработки: «нет» если `weeklyOvertime` = 0, «умеренная» если от 1 до 5, «высокая» если от 6 до 10, «критическая» если больше 10. Использует результат `weeklyOvertime`.
29	parseHours отбрасывает часть строки после #. Если среди значений есть 0, average возвращает 0.0.	`addCommentlessLine(String line) → String` – принимает исходную строку табеля и возвращает её без части после символа #. Если символа # нет, возвращает строку без изменений. Удобно для предобработки перед `parseHours`.
30	overtimeDays возвращает количество «рискованных» дней: v > 8 И v – чётное.	`addRiskScore(int[] hours) → int` – возвращает сумму значений часов для «рискованных» дней: дней, где значение больше 8 и при этом чётное. Чем выше `score`, тем больше накопленный риск. Алгоритм: пройти по массиву, для каждого `v > 8 && v % 2 == 0` прибавить `v` к результату.

Принцип всех вариантов: часть А изменяет одно правило или инвариант, часть Б добавляет новый public static метод с предметной логикой анализа табеля.

7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

По итогам выполнения проект должен:

- компилироваться без ошибок;
- проходить все 19 unit-проверок SelfCheck и минимум 4 проверки варианта (23+ PASS);
- корректно разбирать строку табеля и вычислять 4 показателя по правилам R1–R9;
- выбрасывать IllegalArgumentException с указанием позиции при ошибках формата и диапазона;
- data-driven режим обрабатывает все строки sample.csv и выводит PASS для каждой;
- интерактивный режим принимает строки из System.in до ввода exit или пустой строки;
- Activity-диаграмма и диаграмма классов завершены и согласованы с кодом;
- индивидуальный вариант реализован в полном объёме;
- репозиторий GitHub публичный и содержит все обязательные файлы.

Структура репозитория:

```
pr5-<фамилия>-<инициалы>/
├── README.md                    # Инструкция сборки и запуска
├── .gitignore                   # Исключает out/, .idea/, *.iml, *.class
├── src/
│   └── ru/
```

```
│       └── edu/
│           └── pr02/
│               ├── TimesheetAnalyzer.java
│               └── SelfCheck.java
├── data/
│   └── sample.csv
├── diagrams/
│   ├── activity.dot
│   ├── activity.png
│   ├── class.dot
│   └── class.png
└── DECISIONS.md
```

По завершении практической работы № 5 студент должен иметь следующие результаты. Все элементы обязательны – отсутствие любого из них снижает оценку.

№	Элемент	Что именно должно быть готово
1	Работающий проект starter_project	Компилируется командой javac -encoding UTF-8 без ошибок. Все 5 методов TimesheetAnalyzer (parseHours, total, average, overtimeDays, weeklyOvertime) реализованы по правилам R1–R9. SelfCheck расширен до 23+ проверок (19 базовых + минимум 4 проверки варианта, все PASS). Data-driven режим читает data/sample.csv и выводит PASS для каждой строки. Интерактивный режим принимает строки из System.in до ввода exit.
2	Файл DECISIONS.md	Markdown-файл (80–150 строк) в корне репозитория. Содержит: номер варианта и описание частей А и Б; разделы «Решение 1» и «Решение 2» с формулировкой, альтернативой и обоснованием; раздел «Ответы на контрольные вопросы» – 4 ответа по 50–150 слов каждый; раздел «Фрагмент кода для анализа» – 2–4 предложения об ошибке off-by-one.
3	Завершённая Activity-диаграмма	Файлы diagrams/activity.dot и diagrams/activity.png (разрешение ≥1200×800 px). Блоки TODO A1 и A2 заполнены конкретными формулировками действий. Последовательность ветвлений совпадает с фактической реализацией кода.
4	Завершённая диаграмма классов	Файлы diagrams/class.dot и diagrams/class.png (разрешение ≥1200×800 px). Отражает все public-методы TimesheetAnalyzer, включая новый метод из части Б варианта. Несогласованности между диаграммой и кодом не допускаются.
5	Расширенный sample.csv	Файл data/sample.csv в формате <вход>|<ожидание>|<комментарий>. Содержит минимум 13 строк: корректные строки, ошибки формата, диапазона, структуры, leading zero, trailing separator. Дополнен 3–5 строками с логикой индивидуального варианта.
6	Отчёт REPORT.docx (или .pdf)	Файл в корне репозитория, 5–10 страниц. Times New Roman 12 pt, межстрочный 1.5, поля: верхнее/нижнее 2 см, левое 3 см, правое 1.5 см. Содержит 9 разделов: цель работы, описание реализации, индивидуальный вариант, Activity-диаграмма (PNG), диаграмма классов (PNG), результаты SelfCheck (скриншот), фрагмент кода для анализа, ответы на контрольные вопросы, ссылка на GitHub.
7	Публичный репозиторий GitHub	Репозиторий pr5-<фамилия>-<инициалы> с публичным доступом (без авторизации). Содержит: README.md с командой сборки; .gitignore (исключает out/, .idea/, *.iml, *.class); src/ru/edu/pr02/ с TimesheetAnalyzer.java и SelfCheck.java; data/sample.csv; diagrams/ с DOT и PNG; DECISIONS.md; REPORT.docx/pdf. Все коммиты – от студента. Коммиты начинаются с метки PR5:.
8	Публичная ссылка на отчёт в СДО	Указывается в качестве ответа на задание в СДО. Ведёт на отчёт на облачном ресурсе (Яндекс.Диск, Google Drive) или непосредственно на REPORT.docx/pdf в репозитории GitHub.

8. ТАБЛИЦА ОЦЕНИВАНИЯ РЕЗУЛЬТАТОВ РАБОТЫ

Защита проводится в форме демонстрации работоспособности проекта и устного ответа на вопросы (не более 5 минут). За выполнение работы над индивидуальным проектом студент получает до 2,0 баллов максимум. Для оценивания используется таблица из 40 показателей. Каждый показатель – конкретное действие, которое студент должен продемонстрировать в редакторе кода или консоли: открыть файл, показать метод, изменить параметр, запустить проверку, объяснить результат. В четвёртой колонке таблицы указано, что именно нужно показать и объяснить. Опрос проводится выборочно по пунктам таблицы; демонстрация завершается на первом неверном ответе; за студентом закрепляется набранное на этот момент число баллов.

В таблице приведены конкретные умения, демонстрируемые студентом: объяснение концепций, редактирование существующего кода (изменение правил R1–R9), добавление новых методов, создание диаграмм, расширение SelfCheck, анализ чужого кода. Преподаватель выборочно проверяет освоение материала через практические демонстрации.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
1	Корректность	Продемонстрируйте разбор строки 8;9;10;8;8 в массив {8,9,10,8,8}	В редакторе кода: откройте класс TimesheetAnalyzer, покажите метод parseHours. Объясните назначение split, цикла и проверок. Запустите SelfCheck и покажите результат соответствующей проверки.
2	Корректность	Продемонстрируйте использование split(";", -1), измените аргумент и объясните результат	В редакторе кода: покажите вызов split с аргументом -1. Измените аргумент и перезапустите SelfCheck. Объясните, почему изменилось поведение при обработке trailing-разделителя, и зачем нужен именно этот аргумент.
3	Корректность	Продемонстрируйте проверку количества значений (R1), измените ожидаемую длину	В редакторе кода: покажите проверку количества элементов после split. Измените ожидаемое количество и перезапустите SelfCheck. Объясните, при каких входах срабатывает ошибка структуры и какое исключение выбрасывается.
4	Корректность	Продемонстрируйте проверку regex R2, объясните отсечение 08, +8, 8.5	В редакторе кода: покажите регулярное выражение для проверки формата. В интерактивном режиме введите строки с ведущим нулём, знаком и дробным значением. Объясните, какие значения отвергаются и почему.
5	Корректность	Продемонстрируйте проверку диапазона R3 [0, 16], измените границы	В редакторе кода: покажите проверку диапазона значений. Измените верхнюю границу и перезапустите SelfCheck. Объясните, какие значения будут приняты, а какие – отклонены после изменения.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
6	Корректность	Продемонстрируйте обработку значения 17, объясните отличие от 16	В интерактивном режиме введите строку со значением 17. Объясните, почему 16 считается корректным, а 17 – нет. Покажите, в каком сообщении об ошибке это отражается.
7	Корректность	Продемонстрируйте обработку значения 0, объясните семантику (R4)	В интерактивном режиме введите строку со всеми нулями. Объясните, как трактуется значение 0: считается ли оно выходным, участвует ли в расчётах, считается ли переработкой.
8	Корректность	Продемонстрируйте обработку пустой строки, измените вход и объясните результат	В интерактивном режиме введите пустую строку. Объясните, почему возникает ошибка структуры и какой результат возвращает split для пустой строки. Покажите сообщение об ошибке.
9	Корректность	Продемонстрируйте обработку trailing-разделителя 8;8;8;8;8;	В интерактивном режиме введите строку с завершающей точкой с запятой. Объясните, сколько элементов возвращает split и почему возникает ошибка структуры. Сравните поведение split с аргументом -1 и без него.
10	Корректность	Продемонстрируйте обработку ведущего нуля 08;8;8;8;8	В интерактивном режиме введите строку с ведущим нулём. Объясните, какое правило нарушено и почему regex его отвергает. Покажите позицию, указанную в сообщении об ошибке.
11	Корректность	Продемонстрируйте обработку surrounding-пробела 8; 9;10;8;8	В интерактивном режиме введите строку с пробелом после разделителя. Объясните, почему возникает ошибка формата и выполняется ли trim() значений. Объясните, почему выбрана строгая валидация без trim.
12	Корректность	Продемонстрируйте fail-fast стратегию R7, объясните	В редакторе кода: покажите, что происходит при обнаружении первой ошибки в цикле. Введите строку с несколькими ошибочными значениями. Объясните, какая ошибка сообщается первой и почему проверка остальных значений не выполняется.
13	Корректность	Продемонстрируйте нумерацию позиций с 1 (R8), измените вход	В интерактивном режиме введите строку с ошибкой в последней позиции. Объясните, почему в сообщении указывается номер позиции, а не индекс массива. Покажите, как формируется номер позиции в коде.
14	Корректность	Продемонстрируйте классификацию ошибок R9 (формата vs диапазона)	В интерактивном режиме введите строку с дробным значением и строку со значением вне диапазона. Объясните, чем отличаются типы ошибок и почему они классифицируются по-разному. Покажите различие в сообщениях.
15	Структура	Продемонстрируйте декомпозицию: 5 методов по одной ответственности	В редакторе кода: покажите все пять методов TimesheetAnalyzer. Объясните, какую единственную задачу решает каждый метод. Объясните, почему методы не объединены в один большой метод.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
16	Структура	Продемонстрируйте вызов total(hours) из average, объясните DRY	В редакторе кода: покажите, где метод average вызывает total. Объясните, почему логика суммирования не дублируется. Объясните, что произойдёт при изменении формулы суммы – сколько методов придётся править.
17	Структура	Продемонстрируйте вызов total(hours) из weeklyOvertime, объясните DRY	В редакторе кода: покажите, где метод weeklyOvertime вызывает total. Объясните, почему делегирование исключает дублирование. Объясните, как изменение недельной нормы отразится на коде.
18	Структура	Продемонстрируйте защитные условия от null и неверной длины	В редакторе кода: покажите проверки в начале методов вычислений. Объясните, какие условия проверяются и какие исключения выбрасываются. Объясните, почему эти проверки – программные контракты, а не валидация пользовательского ввода.
19	Структура	Продемонстрируйте использование именованных констант для порогов 8, 16, 40	В редакторе кода: покажите объявление констант для пороговых значений. Объясните, почему используются именованные константы, а не «магические числа». Сравните читаемость кода с константами и без них.
20	Структура	Продемонстрируйте длину методов (≤25 строк), объясните	В редакторе кода: покажите самый длинный метод. Объясните, почему методы короткие. Объясните, какие преимущества даёт соблюдение ограничения на длину метода.
21	Структура	Продемонстрируйте отсутствие Stream API и лямбд, объясните запрет	В редакторе кода: покажите, что для обхода массива используются циклы, а не Stream API. Объясните, почему в данной работе запрещены Stream API и лямбда-выражения. Объясните цель запрета.
22	Структура	Продемонстрируйте локализацию изменения варианта в одном методе	В редакторе кода: покажите, какой метод изменён в рамках части A варианта. Объясните, почему изменение затронуло только этот метод. Объясните, как декомпозиция помогает локализовать изменения.
23	Диаграммы	Продемонстрируйте Activity-диаграмму, объясните TODO A1	Откройте activity.png. Покажите блок TODO A1 с заполненной формулировкой. Объясните, какому фрагменту кода соответствует этот блок и почему формулировка соответствует принципам построения диаграммы.
24	Диаграммы	Продемонстрируйте Activity-диаграмму, объясните TODO A2	Откройте activity.png. Покажите блок TODO A2 с заполненной формулировкой. Объясните, какому фрагменту кода соответствует этот блок и почему формулировка не является копией кода.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
25	Диаграммы	Продемонстрируйте соответствие Activity-диаграммы коду	Откройте activity.png. Для предложенного преподавателем входа покажите путь исполнения по диаграмме. Объясните, какие блоки диаграммы соответствуют каким строкам кода.
26	Диаграммы	Продемонстрируйте диаграмму классов с 5 методами TimesheetAnalyzer	Откройте class.png. Покажите класс TimesheetAnalyzer с его методами. Объясните, почему класс не имеет полей. Покажите связь с классом SelfCheck.
27	Диаграммы	Продемонстрируйте новый метод из части Б на диаграмме классов	Откройте class.png. Покажите добавленный метод в классе TimesheetAnalyzer. Объясните его сигнатуру и назначение. Объясните связь с существующими методами.
28	Диаграммы	Продемонстрируйте читаемость PNG (≥1200×800), объясните инструмент	Откройте activity.png и class.png. Покажите, что изображения читаемы. Объясните, какой инструмент использовался для создания диаграмм и как выполнялся экспорт в PNG.
29	Проверки	Продемонстрируйте 19 unit-проверок SelfCheck, все PASS	Запустите SelfCheck. Покажите вывод с количеством прошедших проверок. Объясните, как проверки распределены по методам. Объясните, какие правила R1–R9 покрывают проверки.
30	Проверки	Продемонстрируйте data-driven режим, прогон sample.csv	Запустите SelfCheck. Покажите вывод data-driven режима. Откройте data/sample.csv и объясните формат строк. Объясните, зачем нужен data-driven режим и в чём его преимущество перед unit-проверками.
31	Проверки	Продемонстрируйте интерактивный режим System.in, объясните	Запустите SelfCheck в интерактивном режиме. Введите корректную и некорректную строки. Объясните, для чего используется интерактивный режим и как он применяется на защите.
32	Проверки	Продемонстрируйте проверку исключений с сообщением	В редакторе кода: покажите try/catch в SelfCheck. Объясните, что проверяется помимо типа исключения. Объясните, почему важна проверка текста сообщения, а не только типа исключения.
33	Проверки	Продемонстрируйте +4 проверки для варианта (2+2)	Запустите SelfCheck. Покажите дополнительные проверки варианта. Объясните, какие проверки добавлены для части А и какие – для части Б. Объясните, какие граничные случаи они покрывают.
34	Проверки	Продемонстрируйте повторный SelfCheck после изменения варианта, отсутствие регрессии	В редакторе кода: внесите правку варианта. Запустите SelfCheck и покажите результат. Объясните, какие ожидания были обновлены и почему это не регрессия.
35	Защита	Продемонстрируйте путь исполнения для 2 разных вводов по Activity-диаграмме	Откройте activity.png. Для двух предложенных преподавателем входов покажите пути исполнения по диаграмме. Объясните каждый переход и соответствие блоку кода.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
36	Защита	Продемонстрируйте объяснение 2 методов, включая новый из части Б	В редакторе кода: объясните назначение и логику двух предложенных преподавателем методов. Объясните, чем новый метод отличается от базовой версии и как он связан с существующими.
37	Защита	Продемонстрируйте объяснение одного граничного условия	Объясните поведение программы для предложенного преподавателем граничного значения. Объясните, почему выбрано строгое или нестрогое неравенство в соответствующей проверке.
38	Защита	Продемонстрируйте внесение небольшого изменения (порог 8→9) за ≤3 мин	В редакторе кода: измените пороговое значение. Обновите ожидания в SelfCheck при необходимости. Запустите SelfCheck и покажите результат. Объясните, почему вынесение порога в константу упрощает изменение.
39	Защита	Продемонстрируйте DECISIONS.md с 4 ответами по 100+ слов	Откройте DECISIONS.md. Покажите четыре ответа на контрольные вопросы. Объясните структуру каждого ответа и какие элементы он содержит.
40	Защита	Продемонстрируйте анализ фрагмента off-by-one (i <= length)	В DECISIONS.md покажите раздел с анализом фрагмента кода. Объясните ошибку, её причину и исправление. Объясните связь ошибки с правилом обработки входных данных.

Структура и требования к отчёту приведены в разделе 11. Требования к составу сдаваемых файлов и Git-репозиторию – в разделе 7.

9. ФРАГМЕНТ КОДА ДЛЯ АНАЛИЗА

В DECISIONS.md студент должен проанализировать фрагмент кода с ошибкой off-by-one. Фрагмент приведён ниже; объясните проблему и покажите исправленный подход в DECISIONS.md в разделе «Фрагмент кода для анализа».

```
// Ошибка off-by-one: условие i <= hours.length
for (int i = 0; i <= hours.length; i++) {
    total += hours[i];
}
```

Формат ответа – 2–4 предложения (50–150 слов). Указать:

- техническую ошибку;
- причину;
- исправленное условие цикла;
- связь с правилом R1 (строка должна содержать ровно 5 значений и обрабатываться без падения).

Этот фрагмент также является предметом проверки на защите: преподаватель может попросить прочитать чужой код с ошибкой off-by-one и объяснить нарушение.

10. КОНТРОЛЬНЫЕ ВОПРОСЫ

По результатам выполнения работы студент должен ответить на четыре контрольных вопроса. Ответы разместить в файле DECISIONS.md в разделе «Ответы на контрольные вопросы». Объём каждого ответа – 50–150 слов. Требования к структуре и содержанию ответов приведены в Приложении В.

Вопрос 1

Какая проверка в этой работе защищает программу от наиболее вероятной ошибки входных данных и где она расположена?

Вопрос 2

Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там?

Вопрос 3

Как Activity-диаграмма отражает один конкретный участок вашего кода?

Вопрос 4

Какое изменение требования в этой работе должно затронуть минимальное количество классов и почему?

№	Вопрос	Формат ответа
1	Какая проверка защищает программу от наиболее вероятной ошибки входных данных и где она расположена?	Название проверки + класс + метод + обоснование, почему эта ошибка наиболее вероятная.
2	Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там?	Название метода/класса + какое правило + обоснование размещения.
3	Как Activity-диаграмма отражает один конкретный участок вашего кода?	Блок на диаграмме + метод в коде + описание соответствия.
4	Какое изменение требования должно затронуть минимальное количество классов и почему?	Пример изменения + сколько классов затронуто + обоснование локализации.

11. СОДЕРЖАНИЕ ОТЧЁТА

Отчёт сдаётся в формате .docx или .pdf. В отчёт студент вставляет ссылку на репозиторий GitHub и заполняет разделы по результатам работы. Отчёт следует разместить на любом доступном облачном ресурсе; в качестве ответа на задание в СДО указать публичную ссылку на отчёт.

11.1. Титульный лист

Титульный лист содержит:

- название работы: «Практическая работа №5. Строки, массивы, циклы и методы обработки данных в Java»;
- ФИО студента, группу;
- номер варианта;
- ссылку на репозиторий GitHub.

11.2. Разделы отчёта

Структура отчёта приведена в таблице. Каждый раздел заполняется по результатам выполнения работы; программный код в отчёт не включается (он доступен в репозитории).

№	Раздел	Содержание
1	Цель работы	Краткая формулировка цели из раздела 1.1.
2	Описание реализации	Краткое описание методов TimesheetAnalyzer (2–3 предложения на метод): что делает, какие правила применяет. Без программного кода.
3	Индивидуальный вариант	Номер варианта. Часть А: что изменено в существующем правиле или методе и почему. Часть Б: какой новый метод добавлен, его назначение и логика.
4	Activity-диаграмма	Изображение итоговой Activity-диаграммы (PNG). Краткое описание (2–3 предложения): какие блоки или переходы изменены или добавлены.
5	Диаграмма классов	Изображение итоговой диаграммы классов (PNG). Краткое описание: какие методы/поля добавлены или изменены.
6	Результаты SelfCheck	Скриншот или текст вывода SelfCheck: все 23+ проверки PASS, data-driven работает. Указать количество проверок.
7	Фрагмент кода для анализа	Ответ на задание раздела 9 (2–4 предложения об ошибке off-by-one).
8	Ответы на контрольные вопросы	4 ответа по 50–150 слов (из раздела 10).
9	Ссылка на репозиторий	Публичная ссылка на GitHub.

11.3. Требования к оформлению

- Формат: .docx или .pdf.
- Шрифт: Times New Roman, 12 pt, межстрочный интервал 1.5.
- Поля: верхнее/нижнее 2 см, левое 3 см, правое 1.5 см.
- Изображения диаграмм – читаемы (≥1200×800 px), с подписями «Рисунок N – ...».
- Объём: 5–10 страниц (без титульного листа и приложений).
- Скриншот вывода SelfCheck – в разделе 6, читаемый.

12. ИТОГ

После выполнения ПР5 учебный проект получает завершённую реализацию 5 методов TimesheetAnalyzer по правилам R1–R9, расширенный SelfCheck с 19 unit-проверками и data-driven режимом чтения sample.csv, завершённые Activity-диаграмму и диаграмму классов, согласованные с фактической реализацией.

Строка → split(";", -1) → regex + диапазон → int[] → total/average/overtimeDays/weeklyOvertime → отчёт

Реализован индивидуальный вариант: часть А (корректировка существующего правила) и часть Б (новый метод с предметной логикой), локализованные в классе TimesheetAnalyzer. SelfCheck обновлён 4 дополнительными проверками, sample.csv – 3–5 строками с новой логикой.

Таким образом, ПР5 закрепляет базовые средства Java (String, split, массивы примитивов, циклы, методы) на прикладной задаче анализа табеля учёта рабочего времени и подготавливает студента к изучению объектно-ориентированных средств языка в последующих работах.

ПРИЛОЖЕНИЕ А. РАСШИРЕННЫЙ SAMPLE.CSV

Полный пример data/sample.csv приведён ниже. Строки, начинающиеся с символа #, – комментарии и игнорируются при чтении. Формат каждой строки: <вход>|<ожидание>|<комментарий>. Поле <ожидание> содержит OK:sum=S,avg=A.AA,od=O,wo=W для корректных строк или ERROR:<описание> для ошибок.

```
# Расширенный sample.csv -- контрольные значения
# Формат: <вход>|<ожидание>|<комментарий>
8;8;8;8;8|OK:sum=40,avg=8.00,od=0,wo=0|базовая корректная строка
9;8;7;10;8|OK:sum=42,avg=8.40,od=2,wo=2|два дня с переработкой
10;10;10;10;10|OK:sum=50,avg=10.00,od=5,wo=10|все дни переработки
0;0;0;0;0|OK:sum=0,avg=0.00,od=0,wo=0|нижняя граница
16;16;16;16;16|OK:sum=80,avg=16.00,od=5,wo=40|верхняя граница
EMPTY|ERROR:структура|пустая строка
8;8;8;8|ERROR:структура|четыре значения
8;8;17;8;8|ERROR:позиция 3|значение вне диапазона
8;abc;8;8;8|ERROR:позиция 2|не число
8;8.5;8;8;8|ERROR:позиция 2|дробное значение
08;8;8;8;8|ERROR:позиция 1|ведущий ноль
8;8;8;8;8;|ERROR:структура|trailing-разделитель
8;;9;10;8|ERROR:позиция 2|пустое поле
8; 9;10;8;8|ERROR:позиция 2|surrounding-пробел
```

При чтении: для каждой строки вызывается parseHours и остальные методы; результат сравнивается с ожиданием. Строки с ERROR ожидаем выброс IllegalArgumentException с указанием позиции или структуры. Студент может расширять sample.csv собственными тест-кейсами для проверки индивидуального варианта (часть А и часть Б). На защите преподаватель использует собственный набор входных данных, поэтому решение, подогнанное только под открытый sample.csv, не считается завершённым.

ПРИЛОЖЕНИЕ Б. ПРИМЕР ЗАПИСИ ВАРИАНТА В DECISIONS.MD

```
# DECISIONS.md

## Вариант
Номер варианта: 7
Часть А: Прогрессивная weeklyOvertime – если sum > 48, = (sum – 48) × 2;
если 40 < sum ≤ 48, = sum – 40; иначе 0.
Часть Б: addFatigueIndex(int[]) → int – индекс усталости =
overtimeDays × 3 + (sum > 40 ? 2 : 0).

## Решение 1
```

```
Формулировка, альтернатива, обоснование

## Решение 2
Формулировка, альтернатива, обоснование

## Ответы на вопросы методички
### Вопрос 1
### Вопрос 2
### Вопрос 3
### Вопрос 4

## Фрагмент кода для анализа
```

ПРИЛОЖЕНИЕ В. ШАБЛОН ОТЧЁТА

Ниже – структура отчёта (раздел 14). Скопируйте в .docx или .pdf и заполните.

```
ПРАКТИЧЕСКАЯ РАБОТА №5
Строки, массивы, циклы и методы обработки данных
Студент: _____ Группа: _____
Номер зачётной книжки: _____
Номер варианта: _____
Ссылка на репозиторий GitHub: _____
Дата сдачи: _____

1. ЦЕЛЬ РАБОТЫ
[Краткая формулировка цели]

2. ОПИСАНИЕ РЕАЛИЗАЦИИ
parseHours: [2–3 предложения]
total: [2–3 предложения]
average: [2–3 предложения]
overtimeDays: [2–3 предложения]
weeklyOvertime: [2–3 предложения]

3. ИНДИВИДУАЛЬНЫЙ ВАРИАНТ №____
Часть А (корректировка): [что изменено и почему]
Часть Б (новый метод): [название метода, назначение, логика]

4. ACTIVITY-ДИАГРАММА
[Изображение PNG]
Описание изменений: [2–3 предложения]

5. ДИАГРАММА КЛАССОВ
[Изображение PNG]
Описание изменений: [какие методы добавлены/изменены]

6. РЕЗУЛЬТАТЫ SELFCHECK
[Скриншот или текст вывода SelfCheck]
```

```
Количество проверок: ____

7. ФРАГМЕНТ КОДА ДЛЯ АНАЛИЗА
[2-4 предложения об ошибке off-by-one]

8. ОТВЕТЫ НА ВОПРОСЫ МЕТОДИЧКИ
Вопрос 1: [50-150 слов]
Вопрос 2: [50-150 слов]
Вопрос 3: [50-150 слов]
Вопрос 4: [50-150 слов]

9. ССЫЛКА НА РЕПОЗИТОРИЙ
https://github.com/_____
```

ПРИЛОЖЕНИЕ Г. ТРЕБОВАНИЯ К ОТВЕТАМ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Студент должен заполнить следующие формулировки в `DECISIONS.md` (или в отдельном файле `REFLECTIONS.md`). Объём каждого ответа – 50–150 слов.

Вопрос 1

Какая проверка в этой работе защищает программу от наиболее вероятной ошибки входных данных и где она расположена?

Ожидаемые элементы ответа: (1) название проверки (напр., «проверка количества значений» или «проверка диапазона»); (2) точное место в коде – класс, метод, строка; (3) обоснование, почему именно эта ошибка «наиболее вероятная» (напр., пользователь часто опечатывается, вводит лишний `;`, забывает одно значение).

Вопрос 2

Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там?

Ожидаемые элементы: (1) название метода/класса; (2) какое правило (напр., «формула недельной переработки max(0, sum − 40)»); (3) обоснование размещения (напр., «метод `weeklyOvertime`, потому что это чистая функция без состояния, легко тестировать и переиспользовать»).

Вопрос 3

Как Activity-диаграмма отражает один конкретный участок вашего кода?

Ожидаемые элементы: (1) указание блока на диаграмме (напр., «TODO A1»); (2) указание метода в коде (напр., «цикл for в `parseHours`»); (3) описание соответствия – какие строки кода реализуют этот блок диаграммы.

Вопрос 4

Какое изменение требования в этой работе должно затронуть минимальное количество классов и почему?

Ожидаемые элементы: (1) пример изменения (напр., «изменение порога дневной переработки с 8 на 9»); (2) сколько классов/методов затронуто (напр., «один метод `overtimeDays`»); (3) обоснование – почему изменение локализовано (напр., «порог вынесен в константу `DAILY_NORMAL_HOURS`, изменение константы не требует правки логики»).